



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

Egypt University of Informatics
Computer and Information Systems

Restaurants In Egypt ChoiceX

Submitted by: Farida Osama / 24-101513

Arwa Ayman / 24-101186

Ali Walid / 24-101042

Hana Kamal / 24-101000

Introduction

This project is a food recommendation chatbot built using scala. The chatbot is designed to talk to users, understand their preferences and suggest restaurants in Egypt based on what they like such as cuisine, budget, city, and vibe.

The system also keeps track of the conversation, updates user preferences during the chat and gives personalized recommendations. It combines object-oriented and functional programming concepts to make the chatbot more interactive and easier to organize.

Motivation

Finding places to eat in Egypt can be confusing because there are too many options, and people usually have different preferences when it comes to food, budget, and atmosphere. Most of the time, users end up spending a lot of time searching without getting recommendations that really match what they want.

This project was created to make that process easier by building a simple chatbot that understands user preferences and suggests suitable restaurants based on them. The idea is to make food discovery faster, more personal, and more interactive through a conversational system.

Design Choices

The project was divided into multiple classes and objects to make the code more organized and easier to maintain.

1. InputHandler

- Reading user input
- Converts text to lowercase and splits it
- Detects keywords from the input
- Classifies the request type (greeting, recommendation, vibe, preferences)

2. Food Places:

- Represents each restaurant in the dataset
- Stores: name, city, cuisine, budget, rating, best for, vibe
- Load data from a text file
- Converts each line into a foodplace object

3. Users:

- Stores user preferences (city, cuisine, budget, vibe, specialty, rating)
- Updates preferences based on user input
- Clears preferences when needed
- Explains current preferences in a readable format
- Allows personalization of recommendations

4. ChatHistory:

- Stores all conversation between user and chatbot
- Logs each message with sender and type
- Detects repeated queries
- Finds most discussed topic
- Generates conversation summary
- Analyzes user mood based on message content

5. Chatbot:

- Main logic of the system
- Generates responses based on input type
- Updates user preferences during conversation
- Filters restaurants to generate recommendations
- Interacts with ChatHistory for analysis
- Controls the chat flow

6. Main:

- Loads food places data from file
- Creates User object
- Creates Chatbot object
- Starts the conversation loop

Implementation Details

The system has main components like a console interface, an input handler, and a recommendation system. The user talks to the chatbot through the console, and the input is processed to understand what they want and extract their preferences. The recommendation system then uses higher-order functions to filter the FoodPlace data based on things like rating, budget, city, and cuisine.

The conversation is stored using the ChatHistory class so the chatbot can track previous messages, detect repeated questions, summarize the discussion, and understand the user's mood. The User class is also used to save and update preferences during the conversation for better recommendations.

Testing Strategy and Results

The system was tested by running user inputs to make sure each part works correctly. This included testing greetings, asking for recommendations, updating preferences, checking history, and exiting the chatbot. The input handler was able to correctly identify most user intents based on keywords. For example, when the user types "hello", it is classified as a greeting, and when the user types "recommend me a restaurant", it is classified as a recommendation request.

The recommendation system was tested using different combinations of preferences such as city, cuisine, budget, and rating. For example, when the user selects "Cairo" and "Italian cuisine", the system returns matching restaurants from the dataset. The chat history was also tested to ensure messages are stored correctly, repeated questions are detected, and mood analysis works based on user messages like "I love this place" or "this is bad".

Future Work Suggestions

Several improvements can be added in future versions of the project.

- Adding machine learning for smarter recommendations
- Supporting Arabic language input
- Creating a graphical user interface
- Connecting the chatbot to an online database
- Adding support for more cities and restaurants
- Ranking recommendations based on restaurant ratings

These improvements would make the chatbot more flexible and friendly.

Conclusion

This project presents a food recommendation chatbot developed using Scala that interacts with users, understands their preferences, and suggests suitable restaurants based on criteria such as city, cuisine, budget, and vibe.

The chatbot tracks conversations, updates user preferences during the chat, and also looks at user mood to make interactions more personal. Overall, it shows how object-oriented and functional programming can work together to build a simple but useful recommendation system.

